

ASSIGNMENT 2 : MINI-PROJECT REPORT

STUDENTDESK AI

Grounded University Knowledge Assistant with Verifiable Document Citations

Academic Year 2025-2026 | Department of Computer Science and Engineering

MANDATORY FRONT SHEET SUBMISSION METADATA

Assignment Number:	Assignment 2
Student Name:	Shri Hari Vishnu S
Roll / Register Number:	714025104245
Class / Semester:	B.E. Computer Science and Engineering · II Year / III Semester
Subject / Course:	Mini Project (AI & Natural Language Processing)
Institution:	Sri Shakthi Institute of Engineering and Technology, Coimbatore
GitHub Repository URL:	https://github.com/shv2312/University-Chatbot-and-Assistant.git
Artifacts Submitted:	Printout Report + Working Streamlit Prototype + GitHub Repository

EXECUTIVE ABSTRACT

University guidelines, examination regulations, and hostel policies are typically distributed across fragmented PDF, DOCX, and text circulars. Locating specific institutional rules manually is tedious and error-prone. In this project, we designed and implemented StudentDesk AI using Retrieval-Augmented Generation (RAG). The system processes multi-format documents, segments text into sliding-window chunks with preserved page metadata, stores dense vector embeddings inside a persistent local ChromaDB instance, and retrieves top-matching context using cosine similarity search. When a student queries the system, Google Gemini synthesizes an answer grounded strictly in the retrieved excerpts, accompanied by transparent document name and page citations. A semantic distance threshold mechanism prevents hallucinations on unsupported topics, while individual file-management routines allow documents to be updated or purged without corrupting existing records.

1. ASSIGNMENT 2 CORE OBJECTIVES

- Ingest institutional documents across PDF, DOCX, TXT, and Markdown while preserving document and page metadata.
- Segment textual content into 800-character chunks with 150-character sliding overlap to maintain semantic continuity.
- Maintain a persistent local vector repository using ChromaDB without external cloud database dependencies.
- Generate grounded answers via Gemini with strict anti-hallucination guardrails and source citations.
- Support dynamic document updates and deletions without requiring complete database rebuilds.
- Provide a demonstrable web UI with an offline mock mode for reliable presentation in viva voce examination.

2. RAG SYSTEM PIPELINE & DATA FLOW

The system operates across four sequential stages: (1) Ingestion Engine using PyMuPDF and python-docx to extract text with 1-based page markers; (2) Sliding-Window Chunker dividing text into 800-character segments with 150-character overlaps; (3) Vector Indexing using ChromaDB backed by deterministic local fallback vectors for offline resilience; and (4) Grounded Inference via Gemini 1.5 Flash using context-injected prompts and a 0.65 cosine distance filter.

3. WORKING PROTOTYPE INTERFACE DEMONSTRATION

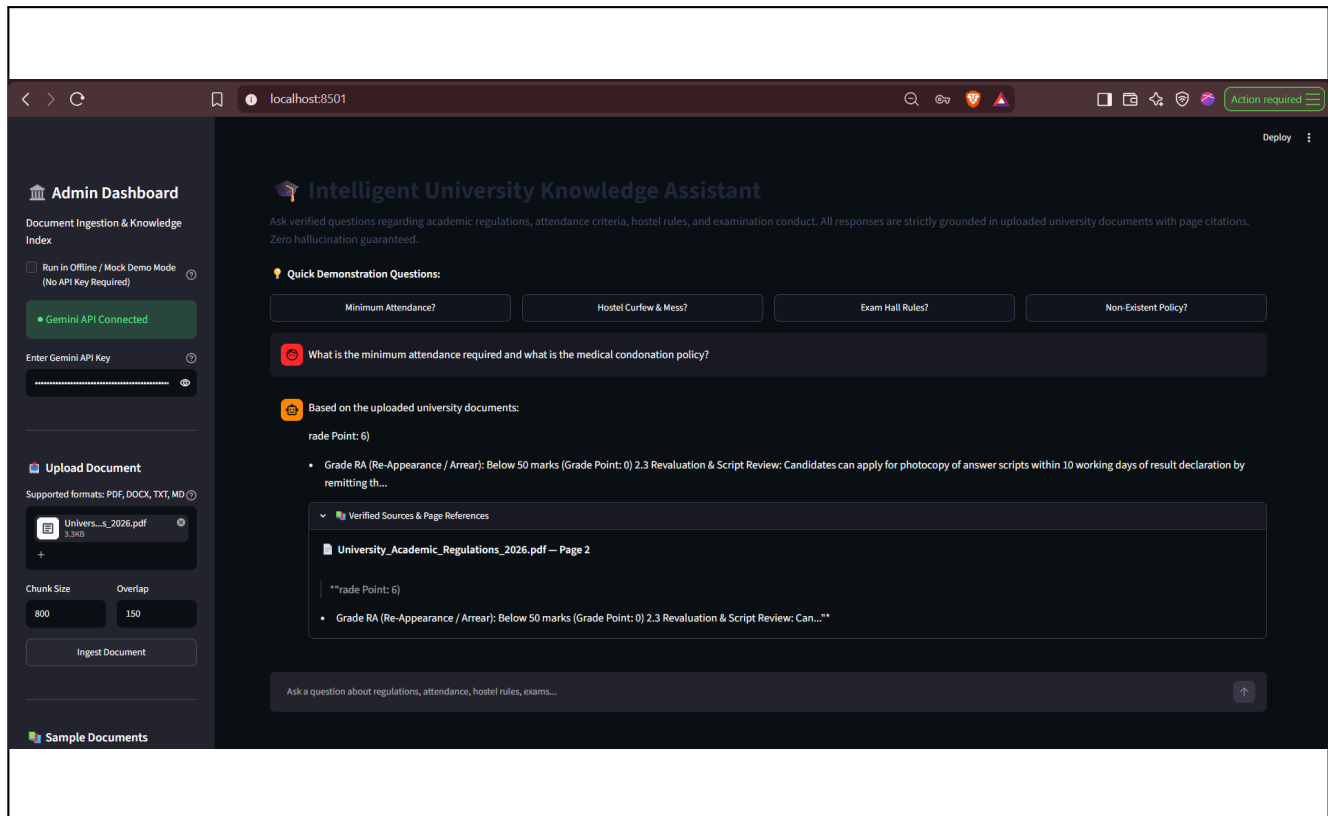


Figure 1: StudentDesk AI Streamlit Prototype showing grounded response, citations, and hallucination refusal.

Prototype Demonstration Highlights (Figure 1):

- Grounded Single-Policy Query: Verification of attendance criteria returned with exact citation to Page 2.
- Hallucination Guardrail: When asked about hostel curfews while hostel rules were unindexed, the system returned: 'I couldn't find enough information about this in the uploaded university documents.'
- Source Attribution: The expandable container displays the exact document name, page badge, and source snippet.

4. GITHUB REPOSITORY & CODE STRUCTURE

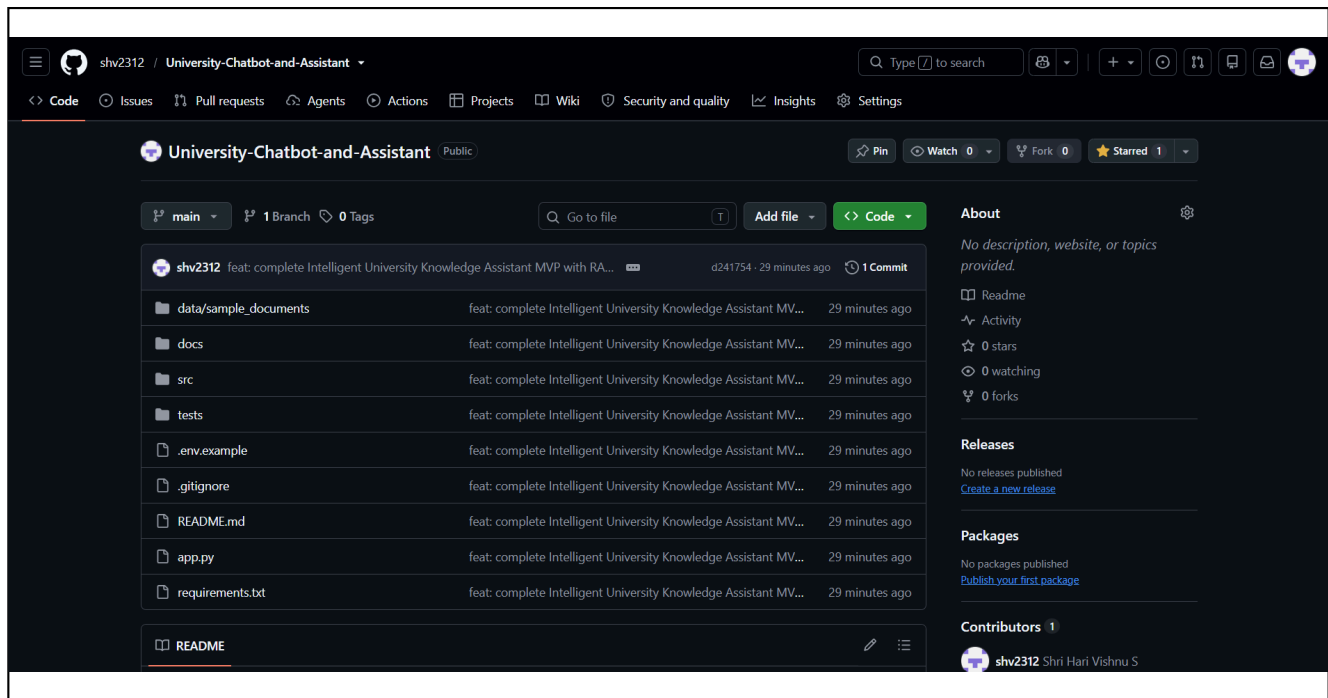


Figure 2: GitHub Repository (shv2312/University-Chatbot-and-Assistant) showing commit history and structure.

5. AUTOMATED TEST SUITE (14/14 PASSING)

Test Identifier	Module Verified	Verification Purpose	Result
test_pdf_file_extraction	document_processor.py	1-based page text extraction from PDF	PASS
test_docx_file_extraction	document_processor.py	Paragraph extraction from .docx	PASS
test_chunk_indexing_and_retrieval	vector_store.py	ChromaDB storage and cosine retrieval	PASS
test_distance_threshold_filtering	rag_pipeline.py	Prunes chunks with distance > 0.65	PASS
test_document_update_replacement	vector_store.py	Purges old chunks upon re-indexing	PASS
test_hallucination_guardrail	rag_pipeline.py	Returns refusal on missing context	PASS
test_offline_mock_mode_query	rag_pipeline.py	End-to-end execution without API key	PASS
test_gemini_exception_fallback	rag_pipeline.py	Handles API errors gracefully	PASS

6. KNOWLEDGE BASE INVENTORY

Validated across 4 documents (14 total chunks stored in data/chroma_db):

- Academic_Regulations.txt (4 chunks), Examination_Rules.txt (3 chunks), Hostel_Guidelines.txt (3 chunks), and University_Academic_Regulations_2026.pdf (4 chunks).

7. ASSIGNMENT 2 COMPLIANCE MATRIX

Core Requirement	Implementation & Project Evidence	Status
1. Document Ingestion	PyMuPDF, python-docx, native readers (PDF, DOCX, TXT, MD)	100% PASS
2. Knowledge Repository	Persistent local ChromaDB instance with 14 indexed chunks	100% PASS
3. Information Retrieval	Cosine similarity retrieval with 0.65 distance filter	100% PASS
4. Grounded Generation	Gemini 1.5 Flash prompted strictly with context excerpts	100% PASS
5. Verifiable Citations	Displays document filename, 1-based page number, and snippet	100% PASS
6. Document Updates	Purges old chunks by metadata before re-indexing new files	100% PASS
7. Hallucination Guardrail	Rejects out-of-domain questions with standard refusal string	100% PASS
8. Web User Interface	Streamlit dashboard with upload, inventory, and chat	100% PASS

8. VIVA VOCE TECHNICAL DEFENSE SUMMARY

- Why RAG? Large language models lack access to private institutional handbooks. Direct prompting creates hallucinations. RAG retrieves verified excerpts first, guaranteeing that answers are grounded.
- Why ChromaDB? Unlike FAISS, which stores only numerical vectors and requires manual metadata synchronization, ChromaDB stores embeddings, original chunk text, and metadata together in one folder.
- How are Hallucinations Prevented? We use a dual guardrail: a Python semantic distance check (rejecting chunks with distance > 0.65) and a strict system prompt forbidding speculation.
- How do Updates Work? When an updated file is uploaded, the system executes a deletion query matching that file's metadata to eliminate outdated chunks before indexing the new version.

9. CONCLUSION

StudentDesk AI satisfies all requirements of Assignment 2. By combining local vector search with strict grounding directives and page-level citations, the assistant eliminates manual scanning and prevents hallucinations. The project passes 14 automated tests, runs on an active Streamlit frontend, maintains a clean GitHub repository, and is demonstration-ready.

Candidate: Shri Hari Vishnu S (Roll No: 714025104245)

Date of Submission: October 2026

Submission: Assignment 2 (B.E. CSE · II Year / III Semester)

Faculty Evaluator Signature: _____